

Advanced Programming

Project assignment – final stage

Final stage

- All students must update the project and *checkout* the `main` *branch*
- One of the students should create a new *branch* from the `main` *branch*
 - *Branch* name: **final**
 - The student who creates the *branch* should *push* it to *GitHub* to make it available to the rest of the group
- The remaining students should update the project and then *checkout* the **final** *branch*

Final stage

- Deadline to perform the *Pull Request* and the *Squash and Merge* (applies to all groups, including working students):

*** 2025.06.02 23:59 ***

- The *Squash and Merge* into the *main* branch must be performed on the *GitHub* website
- The branch must not be deleted after the *Merge* is completed

Final stage

- In this final stage, the following features are to be implemented and/or completed:
 - All functionalities of the chess game
 - Code organization according to the specified requirements
 - Inclusion of unit tests
 - Class documentation using Javadoc
 - Report preparation

Chess game: main features

- The chess game features include:
 - Starting a new game (with player name input)
 - Saving and loading games using serialization
 - Importing (with player name input) and exporting partial games in the specified format
 - Correct game mechanics and rule enforcement
 - End-of-game detection (*checkmate* and *draw*)
 - Special moves (*castling*, *pawn promotion*, and *en passant*)
 - Learning mode with *undo* and *redo* functionalities
 - Audio descriptions of moves

Code organization

- Code organization according to the requirements defined throughout the different stages, including:
 - Separation of data model classes from UI classes
 - The UI must not reference internal model objects
 - The data model must not contain any form of user interaction logic
 - The *Board* class acts solely as a repository of pieces
 - Game mechanics must be handled by the *ChessGame* class, not by *Board*
 - An observable *Facade* class using *Property Change Support*
 - It must not implement any game logic or store any UI-related data
 - A *ModelLog* class implemented as a *Singleton* and observable via *property changes*
 - UI components must follow the *MVC@PA* model
 - Views must be updated within the *update* method, triggered by *Property Change Listener* callbacks

Unit tests

- Prepare unit tests to validate key parts of the chess game data model
 - Include at least 6 varied tests
 - Each test should target a different aspect of the implementation

Javadoc documentation

- Generate complete Javadoc documentation for the following classes:
 - *ChessGame*
 - *ChessGameManager*
 - *ModelLog*
- The documentation only needs to be prepared
 - It does not need to be included in the GitHub submission
 - The group must be able to generate it during the final project presentation

Final report

- The final report must include:
 - A concise description of the main design choices and implementation decisions
 - Diagrams illustrating the design patterns applied in the project
 - A table providing a brief description of the classes used in the program (what they represent and their purpose)
 - A concise description of the relationships between classes (UML diagrams may be used)
 - For each expected feature of the application, an indication of whether it was:
 - Fully implemented
 - Partially implemented (specifying what was actually completed)
 - Not implemented (explaining the reason)

A table may be used to simplify the presentation of this information

Optional extra feature – Board editor

- To recover points not earned in previous stages, the group may choose to implement a partial board editor
 - The editor must allow starting from an empty board and placing any pieces freely (with no restrictions other than the 64 available squares)
 - At the end, it must be possible to export the board using the existing partial game format supported by the application.
- Evaluation conditions
 - This feature is worth up to 0.75 extra points
 - It will only be evaluated if:
 - All core chess game functionalities are fully implemented and functional
 - Code organization requirements are properly met

Final project defense

- Students must prepare for the final defense, which will take place in the days immediately following the submission, mostly during regular class hours
- Defense time slots will be announced in advance on Nónio, and only one group member is required to register the group
- The group must attend the defense fully prepared (project and application open and running)
- Each group will have 7-8 minutes to present all implemented application features
 - Partial game files will be provided in advance to demonstrate specific moves
- Individual questions will follow for each group member
- Defense sessions for students with worker-student status will be scheduled with more spaced-out time slots, to be announced later

Inquéritos sobre PA

- Agradecemos o preenchimento dos seguintes inquéritos:
 - Inquéritos pedagógicos
 - Link: <https://sigq.ipc.pt/inqueritos/alunos/avaliacao>
 - Feedback sobre a metodologia de ensino usada em PA no ano lectivo 2024-25
 - Link: <https://forms.gle/S5wACELhPFZjpJEe6>

Nota: As respostas que não possuírem um identificador válido (distribuído nas aulas práticas) não serão consideradas